

Texas Instruments TMS9918A VDP
Almost complete description including undocumented features

By Sean Young (sean@msxnet.org)
Version 0.4.2 September 2002

In this document I'll try to give a complete description of the TMS9918A in software behaviour and display output. I only know the VDP in the MSX system, in other systems it may have features not described here.

You can always find the latest version of this document at:

<http://www.msxnet.org/tech/>

- 1) Overview
 - 1.1) Colours
 - 1.2) Registers
 - 1.3) External Video
- 2) I/O with the VDP
 - 2.1) Memory access
 - 2.2) Register access
 - 2.3) I/O Ports quirks
 - 2.4) Interrupts
- 3) Display modes
 - 3.1) Mode 0
 - 3.2) Mode 1
 - 3.3) Mode 2
 - 3.4) Mode 3
 - 3.5) Undocumented Mode 1 + 2
 - 3.6) Undocumented Mode 2 + 3
 - 3.7) Other (Mode 1+3 and Mode 1+2+3)
- 4) Sprite system
 - 4.1) Sprite attribute
 - 4.2) Sprite pattern sizes
 - 4.3) Illegal sprites and collisions
- 5) Hardware
 - 5.1) Pin outs
- 6) To-do
- 7) References

1) Overview

The TMS9918a is quite a simple VDP with 16kB or 4kB of video memory. The display always has a resolution of 256 pixels horizontal and 192 pixels vertical. Around this area, there is the border colour (back drop) which can be set in the registers. There also is a sprite subsystem, with maximum of 4 sprites on a horizontal line.

There is a PAL version of the chip, and a NTSC version. The interrupt speed is different: 50Hz for PAL and 60Hz for NTSC. Also the colours are different.

The TMS9918A was used in quite a few systems: MSX1, Coleco, TI99/4A and probably more. I only know it in the MSX1 system.

1.1) Colours

Taken from mess/vidhrdrw/tms9928a.c in MESS, by Raphael Nabet.

The first 3 columns are taken from the TI datasheet (in volts). The next 3 columns are based on this formula:

$$Y = .299*R + .587*G + .114*B \text{ (NTSC)}$$

The coefficients are likely to be slightly different with PAL. I assumed the "zero" for R-Y and B-Y was 0.47V. The last 3 coeffs are the 8-bit values.

Colour	Y	R-Y	B-Y	R	G	B	R	G	B
0 Transparent									
1 Black	0.00	0.47	0.47	0.00	0.00	0.00	0	0	0
2 Medium green	0.53	0.07	0.20	0.13	0.79	0.26	33	200	66
3 Light green	0.67	0.17	0.27	0.37	0.86	0.47	94	220	120
4 Dark blue	0.40	0.40	1.00	0.33	0.33	0.93	84	85	237
5 Light blue	0.53	0.43	0.93	0.49	0.46	0.99	125	118	252
6 Dark red	0.47	0.83	0.30	0.83	0.32	0.30	212	82	77
7 Cyan	0.73	0.00	0.70	0.26	0.92	0.96	66	235	245
8 Medium red	0.53	0.93	0.27	0.99	0.33	0.33	252	85	84
9 Light red	0.67	0.93	0.27	1.13!	0.47	0.47	255	121	120
A Dark yellow	0.73	0.57	0.07	0.83	0.76	0.33	212	193	84
B Light yellow	0.80	0.57	0.17	0.90	0.81	0.50	230	206	128
C Dark green	0.47	0.13	0.23	0.13	0.69	0.23	33	176	59
D Magenta	0.53	0.73	0.67	0.79	0.36	0.73	201	91	186
E Gray	0.80	0.47	0.47	0.80	0.80	0.80	204	204	204
F White	1.00	0.47	0.47	1.00	1.00	1.00	255	255	255

1.2) Registers

The VDP has 8 control registers (0-7) and one status register.

Control Registers:

Reg/Bit	7	6	5	4	3	2	1	0
0	-	-	-	-	-	-	M2	EXTVID
1	4/16K	BL	GINT	M1	M3	-	SI	MAG
2	-	-	-	-	PN13	PN12	PN11	PN10
3	CT13	CT12	CT11	CT10	CT9	CT8	CT7	CT6
4	-	-	-	-	-	PG13	PG12	PG11
5	-	SA13	SA12	SA11	SA10	SA9	SA8	SA7
6	-	-	-	-	-	SG13	SG12	SG11
7	TC3	TC2	TC1	TC0	BD3	BD2	BD1	BD0

Status Register:

	7	6	5	4	3	2	1	0
	INT	5S	C	FS4	FS3	FS2	FS1	FS0

The function of the bits is as follows:

M1,M2,M3	Select screen mode
EXTVID	Enables external video input.
4/16K	Selects 16kB RAM if set. No effect in MSX1 system.
BL	Blank screen if reset; just backdrop. Sprite system inactive
SI	16x16 sprites if set; 8x8 if reset
MAG	Sprites enlarged if set (sprite pixels are 2x2)
GINT	Generate interrupts if set
PN*	Address for pattern name table
CT*	Address for colour table (special meaning in M2)
PG*	Address for pattern generator table (special meaning in M2)
SA*	Address for sprite attribute table
SG*	Address for sprite generator table
TC*	Text colour (foreground)
BD*	Back drop (background). Sets the colour of the border around the drawable area. If it is 0, it is black (like colour 1).
FS*	Fifth sprite (first sprite that's not displayed). Only valid if 5S is set.
C	Sprite collision detected
5S	Fifth sprite (not displayed) detected. Value in FS* is valid.
INT	Set at each screen update, used for interrupts.

The exact meaning of all these bits should become clear in the remainder of

this document.

The bits marked with '-' are "don't care": set or reset, doesn't make any difference. However, to maintain upwards compatibility, you should reset these bits.

1.3) External Video

Tursi <tursi@yiffco.com> writes:

The 9918A processor is capable of genlocking with an external video signal and providing video overlay - this is the purpose of the Transparent color. (I don't know about the 9918 -- perhaps in that chip it *is* B&W, as I've heard of 99/4s (no A) outputting only B&W video, but I haven't seen it so I can not confirm or deny it.)

Anyway, so far as the 9918A goes, the EXTVDP and _RESET/SYNC pins (using the names in your doc) are used to input external video and video sync. When Register 0 bit 0 is reset, the chip performs normally. When it is set, video sync is taken in through the sync pin, and the VDP's video is mixed on top of any video being fed into the EXTVDP pin.

I have experimentally confirmed that this does in fact work. Although the TI-99 is not designed to use this feature, I hacked together a quick circuit many years ago to attempt it. I was not successful in getting the sync to work to my liking (mostly due to inexperience), but I did confirm the operation of this bit as switching the external video on and off.

2) I/O with the VDP

The VDP has two I/O ports (#0 and #1) which can be read or written. In the MSX system, they are mapped to port 98h and 99h. The VDP also has an INT output, which can be used by the CPU to detect screen updates (or vertical blank).

2.1) Memory access

The video memory can't be accessed directly; only through the I/O ports. To the contrary to what many (official!) docs say, there is only one read/write address where the VDP reads or writes in the memory. There also is a read-ahead buffer, which stores one byte. Memory is "read ahead", so when a read is requested, it can give the value immediately.

This memory address (the r/w address) can be set by writing two bytes to port

#1. Despite the fact there is only one address, a distinction is made between setting it for reading or writing (this because of the read-ahead buffer).

The two bytes are formatted as follows:

	7	6	5	4	3	2	1	0
Byte #0	A7	A6	A5	A4	A3	A2	A1	A0
Byte #1	0	R/W	A13	A12	A11	A10	A9	A8

A13-A0 make up the address. R/W specifies whether you want to read or write from that address. It should be 0 for reading, 1 for writing.

What happens internally in the VDP is interesting. This depends on what you set the R/W bit to:

Write (1): Simply set the r/w address to the A* value specified.
Read (0): Read the contents of the A* address from the VRAM and put it
 in the read-ahead buffer. Set the r/w address to A* + 1.

Now that the r/w address is set, we can start reading from or writing to it. After each access it is increased. By reading port #0 we can read from the address, by writing to it we write to the address. What happens internally is:

Read: Return value in read-ahead buffer. Read next value from the r/w address, put it in the read-ahead buffer. Increase the r/w address.
Write: Write back the value to the r/w address, then increase the r/w address. Interestingly, the value written is also stored in the read-ahead buffer.

After 3ffffh (16kB limit) the r/w address wraps to zero.

In 4Kb mode addressing works differently. Charles MacDonald explains:

I wanted to contribute some information to your TMS9918 document. The 4K/16K bit of register #1 tells the TMS9918 how to form addresses when accessing VRAM. However, memory is not really limited to 4K, the full 16K range is addressable. This does have consequences if you write to VRAM in one setting, then change the bit and try to read data back.

The modified addresses look like this:

VDP address	VRAM address	
(Column)	4K mode	8/16K mode
AD0	VA0	VA0
AD1	VA1	VA1
AD2	VA2	VA2
AD3	VA3	VA3

AD4	VA4	VA4
AD5	VA5	VA5
AD6	VA12	VA6
AD7	Not used	Not used
(Row)		
AD0	VA6	VA7
AD1	VA7	VA8
AD2	VA8	VA9
AD3	VA9	VA10
AD4	VA10	VA11
AD5	VA11	VA12
AD6	VA13	VA13
AD7	Not used	Not used

ADx - TMS9928 8-bit VRAM address/data bus
VAx - 14-bit VRAM address that the VDP wants to access

In 4K mode the most significant bit of the DRAM row and column contains the high-order address bits (VA12/VA13). Normally you'd think these bits would be forced to zero, thereby limiting VRAM to 4K and making it appear mirrored throughout the rest of the 16K space.

This has to do with how the cells in a DRAM chip are arranged, they are in a 64x64 array for 4Kx1, and 128x64 or 128x128 for 8Kx1 or 16Kx1. The 4K/16K select bit determines the 'width', meaning how many bits are allocated for the row and column addresses. (mainly 64 versus 128)

I had tested and determined these results from a TMS9929A in an old Sega personal computer, which used eight 16Kx1 chips for 16K of VRAM. I'm fairly sure it also applies to the TMS9918 and TMS9928 as well.

2.2) Register access

The status register can only be read; the control registers can't be read so they can only can be written.

By reading port #1, you get the value of the status register. After reading it, bit 7 (INT bit) and bit 5 (C bit) are reset.

By writing to port #1, a control register is set. Two bytes must be written to port #1:

	7	6	5	4	3	2	1	0
Byte #0	V7	V6	V5	V4	V3	V2	V1	V0
Byte #1	1	?	?	?	?	R2	R1	R0

The V* bits make up the value written to the register, which is specified with the R* bits. Bits 6-3 of the second byte are "don't care": it doesn't matter what value you set them. Officially, they must be 0. It is best to set them to 0. The upwards TMS9918A compatible V9938 and V9958 have more registers and so these bits do have meaning. So, for compatibility, set them to 0. Bit 7 must be set, to indicate you want to write to a register; otherwise you're changing the r/w address.

2.3) I/O Ports quirks

As you've probably noticed, writes to port #1 must always go in pairs of two bytes. Suppose you would write one byte to port #1, and let normal programming continue .. you'd assume that the byte order of any port #1 write is wrong and strange things happen. But that isn't true.

What happens internally is this. After writing the first byte to port #1, a flag is set to indicate that the next byte written to port #1 is the second of the pair and completes the register write or r/w address set. This flag is reset after that.

But this flag is also reset whenever you read port #1, or read or write port #0. This probably done to prevent an accidental single write to port #1 mess up the whole programming (which without this setup, would require a hard reset or another write to port #1 to return to normal operation).

So, in between writing two bytes to port #1, never access any other port of the VDP. This is why interrupts must be disabled during writing to port #1: the interrupt service routine most likely accesses the VDP (status register read for example) and that will mess it up.

2.4) Interrupts

The VDP can generate interrupts for the CPU. It works as follows: at the end of the last line of the screen (after vertical line 192, last of the drawable screen; at the beginning of the bottom part of the backdrop) bit 7 of status register is set.

If bit 7 of the status register is set (INT), and bit 5 of control register #1 is set (GINT), the INT output line is low (requesting an interrupt), otherwise it is high (not requesting an interrupt).

So if bit 5 of control register #1 is reset, interrupts are never requested.

After the status register is read, bit 7 is reset. The INT line goes high again, so it isn't requesting an interrupt anymore. That's why this register

must always be read in an interrupt routine: otherwise the INT is remains low and the interrupt service routine is called again and again.

Another interesting fact is this: if bit 5 of control register #1 is reset, and you set it while bit 7 of the status register is set, the INT line goes low immediately and an interrupt is requested. This is important for emulators too: if this isn't emulated properly, the MSX game Track & Field 1 and 2 slows down considerably.

3) Screen modes

There are 4 official screen modes but 3 mode bits (8 possibilities). The modes define how the screen is built up. In some modes, the sprite subsystem is active which is described in section 4).

All screen modes are character based.

Screen mode x simply means the respective Mx bits are set. Mode 0 indicates none of the M* bits are set.

3.1) Mode 0

This screen mode is known as SCREEN 1 in MSX BASIC, and GRAPHIC 1 in the V9938 Technical Data Book.

Like most screen modes, it has width 32 characters and 24 characters vertical. The sprite subsystem is active.

Since the size is 32 x 24, the Pattern Name Table (PN) has $32 \times 24 = 768$ elements, which are bytes referring to the Pattern Generator Table (PG) and Colour Table (CT).

The first character is the top-left one, the second the one to the right of it. Character no. 32 the first character of the second row.

Each character is 8 x 8. The character number refers to an character pattern in the PG, which has 256 characters patterns. Each pattern is 8 bytes, so the entire PG is $256 \times 8 = 2048$ bytes. Each byte is a pixel line of the character (top first). If a bit is set, the foreground colour in the CT is shown, otherwise the background colour in the CT.

The CT sets the colour of the character. Each byte in the CT sets the colour of 8 complete characters, so the CT is 32 bytes. The high four bits set the foreground colour, the low the background colour. If either of these is 0,

it is transparent and the colour of the backdrop (BD) is used. To calculate the element in the CT, divide the character number in PN by 8.

The TC in control register #7 is not used in this mode.

3.2) Mode 1

This mode is known as SCREEN 0 in MSX BASIC, and as TEXT 1 in the V9938 Technical Data Book.

In this mode, the screen has width 40 characters and 24 characters vertical. Only two colours are used, TC and BD from control register #7. Since the characters are 6 pixels wide, the width is $40 \times 6 = 240$. There is a 8 pixels border at the left and right side of the screen (with the colour of the backdrop). The sprite subsystem is inactive.

Since the size is 40×24 , the Pattern Name Table (PN) has $40 \times 24 = 960$ elements, which are bytes referring to the Pattern Generator Table (PG). The Colour Table (CT) is not used.

The first character is the top-left one, the second the one to the right of it. Character no. 40 the first character of the second row.

Each character is 6×8 pixels. The character number refers to a character pattern in the PG, which has 256 characters patterns. Each pattern is 8 bytes, so the entire PG is $256 \times 8 = 2048$ bytes. Each byte is a pixel line of the character (top first). If a bit is set, the TC in control register #7 is shown, otherwise BD in the same register. The two lowest bits of each byte in the PG are ignored.

3.3) Mode 2

This mode is known as SCREEN 2 in MSX BASIC, and as GRAPHIC 2 in the V9938 Data Book. It is the most complex and has the most possibilities of all the modes of the TMS9918A.

Like most screen modes, it has width 32 characters and 24 characters vertical. The sprite subsystem is active.

Since the size is 32×24 , the Pattern Name Table (PN) has $32 \times 24 = 768$ elements, which are bytes referring to the Pattern Generator Table (PG) and Colour Table (CT).

The first character is the top-left one, the second the one to the right of it. Character no. 32 the first character of the second row. There are three

colour and three pattern tables. The character number is as follows:

The first 8 rows: byte from PN + 000h
The middle 8 rows: byte from PN + 100h
The bottom 8 rows: byte from PN + 200h

Each character is 8 x 8. The character number refers to an character pattern in the PG, which has 768 characters patterns (three distinct tables). For the first 8 rows of the screen, the first 256 entries are used. For the second 8 rows, the second 256 patterns, and for the last 8 rows the third 256 patterns (depending on the value of control register #4, see below). So the entire PG is $3 \times 256 \times 8 = 6144$ bytes. Each byte is a pixel line of the character (top first). If a bit is set, the foreground colour in the CT is used, otherwise the background colour in the CT.

Control register #4 which sets the PG address of the PG has a different function in mode 2. Only bit 2, PG13, sets the address of the PG (so it's either address 0 or 2000h). Bits 0 and 1 are an AND mask over the character number. The character number is 0 - 767 (2FFh) and these two bits are ANDed over the two highest bits of this value (2FFh is 10 bits, so over bit 8 and 9). So in effect, if bit 0 of control register #4 is set, the second array of 256 patterns in the PG is used for the middle 8 rows of characters, otherwise the first 256 patterns. If bit 1 is set, the third array of patterns is used in the PG, otherwise the first.

The relation PN -> CT is the same as PN -> GT. Each byte in the CT sets the colour of one pixel line of a character. The high four bits set the foreground colour, the low the background colour. If either of these is 0, it is transparent and the colour of the backdrop (BD) is used.

Also control register #3 has a different meaning. Only bit 7 (CT13) sets the CT address. Somewhat like control register #4 for the PG, bits 6 - 0 are an AND mask over the top 7 bits of the character number.

The TC in control register #7 is not used in this mode.

3.4) Mode 3

This mode is known as SCREEN 3 in MSX-BASIC, and as MULTICOLOR in the V9938 Technical Data Book. In this mode, each 'unit' on the screen is 4 x 4 pixels. In these units all the pixels have the same colour. Unlike all the other modes there is no colour spill (each pixel can have any colour). The sprite subsystem is active.

The characters are 8 x 8 pixels. There are 32 x 24 characters on the screen, so the pattern name table (PN) has 768 elements (bytes) which refer to a

pattern in the pattern generator table (PG). Only two bytes of the 8 bytes in the pattern are used. Which two bytes depends on the vertical row (0 - 23). The address in the PG can be calculated as follows:

$$PG + (\text{byte in PN}) \times 8 + (\text{row AND } 3) \times 2$$

Each character is 4 units. Each byte sets the color of two blocks of 4 x 4 pixels. The lower four bits set the colour of the left block, the highest four bits set the colour of the right block. If either is 0, the block is transparent and the backdrop (BD) is visible.

The TC in control register #7 is not used in this mode, neither is the colour table (CT).

3.5) Undocumented Mode 1 + 2

This mode is very much like Mode 1; the difference is that there can be three different pattern tables, and control register #4 has a different meaning.

The pattern table now has 3 x 256 patterns; the screen is divided in three parts (the top 8 character rows, the middle 8, the bottom 8). Each part can use a different part of the pattern generator table (the first 256, the second 256, the last 256). The top part always uses the first 256 patterns in the PG.

Control register #4:

- bit 7 - 3 : ignored
- bit 2 : PG13; if set, PG = 2000h otherwise 0
- bit 1 : if set, the last 8 rows use the 3rd pattern table otherwise 1st.
- bit 0 : if set, the middle 8 rows use the 2nd pattern table otherwise 1st.

This undocumented mode is not available in the V9938 as far as I know.

3.6) Undocumented Mode 2 + 3

This mode is very much like Mode 3; it differs in the same way that undocumented mode 1 + 2 differs from mode 1. See section above for details.

3.7) Other (Mode 1+3 and Mode 1+2+3)

There are two undocumented modes left: 1+3 and 1+2+3. Something weird happens in both modes: no longer do the tables have any influence on how the screen looks like. The screen simply shows 40 vertical lines of text colour (TC). By the looks of it, it's 4 pixels TC, 2 pixels BD. Like in mode 1, there is a 8 pixel border with colour backdrop (BD) at the left and right of the screen.

It seems possible that memory refresh isn't done any more, but Tursi <tursi@yiffco.com> tested this, and memory was still intact after a couple of frames.

The sprite subsystem is inactive.

4) Sprite system -----

In all modes except when M1 is set or BL is reset, the sprite subsystem is active. Sprites can be either 8 x 8 pixels, or 16 x 16; also these pixels can be enlarged (twice the size; "zoomed").

4.1) Sprite attribute table

32 different sprites can be display at the same time on the same screen, with the exception that no more than 4 four sprites can be displayed on one horizontal pixel line. The location and colour of the sprites is specified in the sprite attribute table (SA). For each of the 32 sprites, there are four bytes.

SA ->	0	Y sprite 0
	1	X sprite 0
	2	sprite pattern 0
	3	colour sprite 0 + EC
	4	Y sprite 1
	5	X sprite 1
	6	sprite pattern 1
	7	colour sprite 1 + EC
	.	
	.	

The values have special meanings. If the Y coordinate is 0, it's displayed on the second pixel line from the top. With value 255 it's exactly on the the top line, and with coordinate 254 the top line isn't visible any more, with 253 to two top lines aren't visible any more, etc.

If Y is 208, that sprite and all following sprites in the table are not displayed.

The X coordinate is simply, as you would expect, the X coordinate from 0 to 255. If bit 7 of the ``colour sprite'' is set (Early Clock -- EC), the sprite is moved 32 pixels to the left; in this way it's possible to partially display the sprite at the left side.

The lower 4 bits of ``colour sprite'' define the colour. Bit 7 is the EC. Bits 6 - 4 should be zero officially, but they are ignored. If the colour is 0, the sprite is transparent (not visible).

4.2) Sprite generator & sizes

These options are in the control register #1. If bit 0 (MAG) is set, the sprites are enlarged. If bit 1 (SI) is set, the sprites are 16 x 16 in size, otherwise 8 x 8.

The sprite pattern defines which pattern to use. In the case of 8 x 8 sprites, there are 8 bytes for the sprite pattern, and there are 256 patterns in the sprite generator table (SG). So simply multiply the sprite pattern by 8 to get the address in the SG. Each of these bytes set the colour of the pixels of the sprite. If a bit is set, the pixel has the colour of the ``colour sprite'' -- unless it is 0, in which case it is transparent.

In the case of 16 x 16, it is slightly more complex. The SG is still 256 x 8 bytes, and each pattern is 32 bytes now. To calculate the address in the SG: ((pattern number) AND 252) x 8. The first 16 bytes set the colour of the left 8 pixels of the sprite, the next 16 bytes set the colour of the right 16 pixels of the sprite. If it is 0, it is transparent.

If bit 0 of control register 1 (MAG) is set, the sprites are enlarged. The starting coordinates are the same, but every pixel of the sprite is 2 x 2 real pixels.

4.3) Illegal sprites & sprite collision

On each horizontal pixel line, only 4 sprites are allowed. Any more than that are not displayed. These sprites that aren't displayed are known as ``illegal sprites''. The first sprite illegal sprite that is detected is put in the status register. Bit 6 (5S) is set, to show an illegal sprite is detected. The sprite number (SA element number) is stored in the FS* bits. Note that even transparent (colour 0) sprites count for illegal sprites, and sprites that aren't displayed because they're of the left or right side of the screen.

If no illegal sprite was found, 5S won't be set and FS* contains the highest sprite number in the SA, either the last (31) or the first with Y coordinate 208 (if it exists). Note that isn't fully checked (but probably not really important).

Remember the screen is built line by line from top to bottom; the first illegal sprite is on the highest pixel line with an illegal sprite.

If anywhere on the screen two or more sprites overlap, the C bit of the status register is set. The colour of the sprite is not important (can even be 0), the pattern is. If a sprite line is not displayed because it is illegal, it does not count for sprite collision. The C bit of the status register is reset after the register is read.

5) Hardware

5.1) Pin-outs

TMS9918A

+-----+			
_RAS	1	40	XLAT2
_CAS	2	39	XLAT1
AD7	3	38	CPUCLK
AD6	4	37	GROMCLK
AD5	5	36	COMVID
AD4	6	35	EXTVDP
AD3	7	34	_RESET / SYNC
AD2	8	33	Vcc
AD1	9	32	RD0
AD0	10	31	RD1
R/_W	11	30	RD2
Vss	12	29	RD3
MODE	13	28	RD4
_CSW	14	27	RD5
_CSR	15	26	RD6
_INT	16	25	RD7
CD7	17	24	CD0
CD6	18	23	CD1
CD5	19	22	CD2
CD4	20	21	CD3
+-----+			

TMS9928A

+-----+			
_RAS	1	40	XLAT2
_CAS	2	39	XLAT1
AD7	3	38	R-Y
AD6	4	37	GROMCLK
AD5	5	36	Y
AD4	6	35	B-Y

AD3	7	34	_RESET / SYNC
AD2	8	33	Vcc
AD1	9	32	RD0
AD0	10	31	RD1
R/_W	11	30	RD2
Vss	12	29	RD3
MODE	13	28	RD4
_CSW	14	27	RD5
_CSR	15	26	RD6
_INT	16	25	RD7
CD7	17	24	CD0
CD6	18	23	CD1
CD5	19	22	CD2
CD4	20	21	CD3
+-----+			

6) To-do

There are some things still need finding out.

- o How to emulate by scanline: what are the exact timings, when are the different tables read.
- o What are the colours in PAL mode.

7) References

MSX Handboek voor gevorderden

A Dutch MSX1 technical manual. It's full of mistakes but very complete.

V9938 MSX-Video Technical Data Book

The be-all, end-all guide to the MSX2 VDP; some things are true for the TMS9918A VDP too.

<http://www.msxnet.org/tech/tmsposting.txt>

A posting by Paul Urbanus (urb@onramp.net) in comp.emulators.misc.
Very interesting stuff about timings and the one r/w address.

http://home.swipnet.se/~w-16418/tech_vdp.htm

This is an expanded version of my original tms9918a document. It's more complete than my old version, but this document should be better. :-)

<http://www.mess.org/>

The MESS emulator, includes a TMS9928A emulator nearly all

undocumented features described here are emulated.

<http://www.spies.com/~arcade/dataSheets/ti/TMS9918.pdf>
TMS9918 Data Book